BGMI Scope Sensitivity Recommender

Android App + PHP/MySQL Backend + GitHub Actions CI/CD — Build Specification for AI Coding Assistant

1. Project Overview

Build a native Android application (Kotlin) that automatically reads a phone's real hardware specifications and recommends BGMI (Battlegrounds Mobile India) camera and ADS sensitivity values, scope-by-scope. A PHP + MySQL backend API stores a phone-spec database and performs the sensitivity calculation. The app should also work with manual phone selection when auto-detection is incomplete.

1.1 Goals

- Auto-detect device specs on first launch (no manual typing required).
- Fetch/compute recommended sensitivity via a PHP backend API.
- Show results per scope: Free Look, TPP/FPP no-scope, Red Dot/Holo/2x, 3x, 4x, 6x, 8x, plus Gyroscope and ADS sensitivity.
- Let the user save multiple devices and view history.
- Ship a signed, installable APK automatically via GitHub Actions on every push/tag.

---

2. Tech Stack

- **Android app:** Kotlin, native Android SDK (no Flutter/React Native).
- **Backend:** Pure PHP (no framework/Composer) + MySQL — matches existing hosting on cPanel/LiteSpeed shared hosting.
- **Networking:** Retrofit or plain HttpURLConnection/OkHttp calling the PHP REST API (JSON).
- **Build/CI:** GitHub Actions, Gradle, auto-signed release APK as a build artifact / GitHub Release.

Note: Do not hardcode dependency/SDK version numbers from training memory — Android tooling (Android Gradle Plugin, Kotlin, compileSdk/targetSdk, Gradle version) changes frequently. **Web search for the current stable versions** of: Android Gradle Plugin (AGP), Kotlin, Gradle wrapper, compileSdk/targetSdk API level, and the actions/setup-java + gradle-build-action GitHub Actions versions, before generating build.gradle files and the workflow YAML. As a starting reference point only (verify before use): AGP was in its 9.x stable line and Kotlin in its 2.2.x stable line as of early-to-mid 2026 — treat these strictly as a floor, not a final answer.

---

3. Android App Specification

3.1 Screens

- **Home / Auto-Detect:** Single prominent button 'Detect My Device'. On tap, reads specs and calls the backend, then navigates to Results.
- **Manual Select:** Fallback screen — searchable dropdown/list of known phone models (pulled from backend) for when auto-detect data is incomplete or the user wants to compare another phone.

- **Results:** Card-style list showing recommended values for: Camera Sensitivity (Free Look, TPP no-scope, FPP no-scope), Red Dot/Holo/2x (TPP & FPP), 3x scope, 4x scope, 6x scope, 8x scope, ADS Sensitivity, and Gyroscope (per scope, if device has a gyroscope). Include a 'Copy All' and per-row 'Copy' action.
- **Save/History:** List of previously detected/saved devices with timestamp; tap to reopen that result set.
- **About/Feedback (optional v1.1):** Simple form where a user rates whether a recommendation 'felt right', stored for future tuning.

3.2 Device Spec Detection (DeviceSpecsHelper)

Collect the following on-device, without any manual input:

- Screen size in inches — derive from `DisplayMetrics.widthPixels/heightPixels` and `xdpi/ydpi`.
- Screen resolution and density (DPI) — `DisplayMetrics.densityDpi`.
- Refresh rate — `Display.getRefreshRate()` / `Display.getSupportedModes()` on API 23+.
- Device model/manufacturer — `Build.MANUFACTURER`, `Build.MODEL`.
- Android version / API level — `Build.VERSION.SDK_INT`.
- Touch sampling rate — **not exposed by any public Android API**; must come from the backend's phone-spec database, matched by model. If the model is unknown, fall back to a sane default and mark the result as an estimate in the UI.

3.3 App → Backend Contract

The app sends detected specs as JSON to the backend and renders whatever JSON comes back. Keep the app free of sensitivity-calculation logic — all formulas live server-side so they can be tuned without an app update.

```
POST /api/calculate-sensitivity
Content-Type: application/json

{
  "model": "Manufacturer ModelName",
  "screen_size_inches": 6.67,
  "density_dpi": 440,
  "refresh_rate_hz": 120,
  "android_sdk_int": 34
}

Response 200:
{
  "matched_in_db": true,
  "is_estimate": false,
  "sensitivities": {
    "camera": { "free_look": 45, "tpp_no_scope": 92, "fpp_no_scope": 95 },
    "red_dot_holo_2x": { "tpp": 62, "fpp": 65 },
    "scope_3x": 28,
    "scope_4x": 22,
    "scope_6x": 17,
    "scope_8x": 11,
    "ads_sensitivity": 55,
    "gyroscope": { "3x": 120, "4x": 90, "6x": 55, "8x": 35 }
  }
}
```

---

4. Backend API (PHP + MySQL)

4.1 Endpoints

- `GET /api/phones?search=xyz` — search phone-spec database for manual-select autocomplete.
- `GET /api/phone-specs?model=xyz` — fetch stored specs for one exact model.
- `POST /api/calculate-sensitivity` — accepts detected/selected specs, returns recommended sensitivity JSON (see contract above).
- `POST /api/feedback` (optional v1.1) — accepts a device id + up/down or 1-5 rating on a recommendation, for future formula tuning.

4.2 MySQL Schema (starting point)

```
CREATE TABLE phones (
  id INT AUTO_INCREMENT PRIMARY KEY,
  manufacturer VARCHAR(64),
  model VARCHAR(128) UNIQUE,
  screen_size_inches DECIMAL(4,2),
  density_dpi INT,
  refresh_rate_hz INT,
  touch_sampling_rate_hz INT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE feedback (
  id INT AUTO_INCREMENT PRIMARY KEY,
  phone_id INT,
  rating TINYINT,
  scope VARCHAR(32),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (phone_id) REFERENCES phones(id)
);
```

4.3 Calculation Logic

Start with the known-good baseline values already validated for a 6.5″ reference device (see table below), then scale by a screen-size factor and a refresh-rate factor. Keep this formula isolated in one PHP function so it is easy to retune later from feedback data.

Setting	Baseline (6.5" ref device)
Free Look	45
TPP no-scope	90
FPP no-scope	95
Red Dot / Holo / 2x	60-65
3x scope	28
4x scope	22
6x scope	17
8x scope	11

```
// simplified example formula
$screen_factor = $screen_size_inches / 6.5;
$refresh_factor = $refresh_rate_hz >= 90 ? 1.05 : 1.0;
$final_value = round($baseline_value * $screen_factor * $refresh_factor);
```

---

5. GitHub Actions — Auto Build Signed APK

On every push to main (or on a version tag), GitHub Actions should build a release APK and sign it automatically using a keystore stored in GitHub Secrets, then attach the APK to the workflow run (and optionally publish a GitHub Release).

5.1 One-time setup (manual, before first run)

- Generate a release keystore locally: `keytool -genkey -v -keystore release.keystore -alias appkey -keyalg RSA -keysize 2048 -validity 10000`.
- Base64-encode it: `base64 release.keystore > keystore_base64.txt`.
- Add these repo secrets in GitHub → Settings → Secrets and variables → Actions: `KEYSTORE_BASE64`, `KEYSTORE_PASSWORD`, `KEY_ALIAS`, `KEY_PASSWORD`.

5.2 Workflow outline

```
name: Build Signed APK

on:
  push:
    branches: [ main ]
  workflow_dispatch: {}

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up JDK
        uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: '17'   # verify current recommended JDK for chosen AGP/Kotlin

      - name: Decode keystore
        run: echo "${{ secrets.KEYSTORE_BASE64 }}" | base64 -d > app/release.keystore

      - name: Build release APK
        run: ./gradlew assembleRelease
        env:
          KEYSTORE_PASSWORD: ${{ secrets.KEYSTORE_PASSWORD }}
          KEY_ALIAS: ${{ secrets.KEY_ALIAS }}
          KEY_PASSWORD: ${{ secrets.KEY_PASSWORD }}

      - name: Upload APK artifact
        uses: actions/upload-artifact@v4
        with:
          name: app-release-signed
          path: app/build/outputs/apk/release/*.apk
```

Note: Verify the current major version numbers for `actions/checkout`, `actions/setup-java`, and `actions/upload-artifact` via web search before finalizing this YAML — GitHub Actions marketplace versions change over time and old major versions can be deprecated.

5.3 app/build.gradle signing block (reference shape)

```
android {
    signingConfigs {
        release {
```

```
            storeFile file("release.keystore")
            storePassword System.getenv("KEYSTORE_PASSWORD")
            keyAlias System.getenv("KEY_ALIAS")
            keyPassword System.getenv("KEY_PASSWORD")
        }
    }
    buildTypes {
        release {
            signingConfig signingConfigs.release
            minifyEnabled true
            proguardFiles getDefaultProguardFile('proguard-android-optimize.txt'), 'proguard-rules.pro'
        }
    }
}
```

---

6. Deliverables Checklist

- Android Studio project (Kotlin) with the 4 screens above and DeviceSpecsHelper.
- PHP backend with the 3-4 endpoints above, connected to MySQL.
- MySQL schema/migration script, pre-seeded with at least 30-50 common phone models (screen size, density, refresh rate, and — where knowable — touch sampling rate).
- GitHub Actions workflow producing a signed, installable release APK on every push to main.
- Short README covering: how to configure the backend base URL in the app, how to add/rotate the signing keystore secrets, and how to add new phones to the database.

End of specification.